



Classes and Objects



OOP, Defining a Class

- Python was built as a procedural language
 - OOP exists and works fine, but feels a bit more "tacked on"
- Declaring a class:

```
class name:  
    statements
```

Fields

name = value

- Example:

```
class Point:
    x = 0
    y = 0

# main
p1 = Point()
p1.x = 2
p1.y = -5
```

point.py

```
1 class Point:
2     x = 0
3     y = 0
```

- can be declared directly inside class (as shown here) or in constructors (more common)

Using a Class

```
import class
```

- client programs must import the classes they use

point_main.py

```
1 from Point import *
2
3 # main
4 p1 = Point()
5 p1.x = 7
6 p1.y = -3
7 ...
8
9 # Python objects are dynamic (can add fields any time!)
10 p1.name = "Tyler Durden"
```

Object Methods

def name(self, parameter, ..., parameter):
statements

- self must be the first parameter to any object method
- must access the object's fields through the self reference

```
class Point:  
    def translate(self, dx, dy):  
        self.x += dx  
        self.y += dy  
    ...
```

Write distance, set_location, and distance_from_origin methods.

point.py

```
1 from math import *
2
3 class Point:
4     x = 0
5     y = 0
6
7     def set_location(self, x, y):
8         self.x = x
9         self.y = y
10
11     def distance_from_origin(self):
12         return sqrt(self.x * self.x + self.y * self.y)
13
14     def distance(self, other):
15         dx = self.x - other.x
16         dy = self.y - other.y
17         return sqrt(dx * dx + dy * dy)
```

Calling Methods

- A client can call the methods of an object in two ways:
 - (the value of self can be an implicit or explicit parameter)

1) **object.method(parameters)**

or

2) **Class.method(object, parameters)**

- Example:

```
p = Point(3, -4)
p.translate(1, 5)
Point.translate(p, 1, 5)
```

Constructors

```
def __init__(self, parameter, ...,  
parameter):  
    statements
```

- a constructor is a special method with the name `__init__`
- Example:

```
class Point:  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y  
    ...
```

__str__

```
def __str__(self):  
    return string
```

- converts object to a string
- invoked automatically when str or print is called

Exercise: Write a `__str__` method for Point objects that returns strings like `"(3, -14)"`

```
def __str__(self):  
    return "(" + str(self.x) + ", " + str(self.y) +  
    ")"
```

Complete Point Class

point.py

```
1 from math import *
2
3 class Point:
4     def __init__(self, x, y):
5         self.x = x
6         self.y = y
7
8     def distance from origin(self):
9         return sqrt(self.x * self.x + self.y * self.y)
10
11     def distance(self, other):
12         dx = self.x - other.x
13         dy = self.y - other.y
14         return sqrt(dx * dx + dy * dy)
15
16     def translate(self, dx, dy):
17         self.x += dx
18         self.y += dy
19
20     def __str__(self):
21         return "(" + str(self.x) + ", " + str(self.y) + ")"
```

INTRODUCTION

- **FILE HANDLING** is a mechanism by which we can **read data** of disk files in python program or **write back data** from python program to disk files.
- So far in python program the standard **input** was from **keyboard** an **output** was going to monitor i.e. no **data is stored permanently** and **entered data is present as long as program is running**.
- But **file handling** allows us to **store data** entered through python program permanently in disk file and later on we can read back the data.

Files

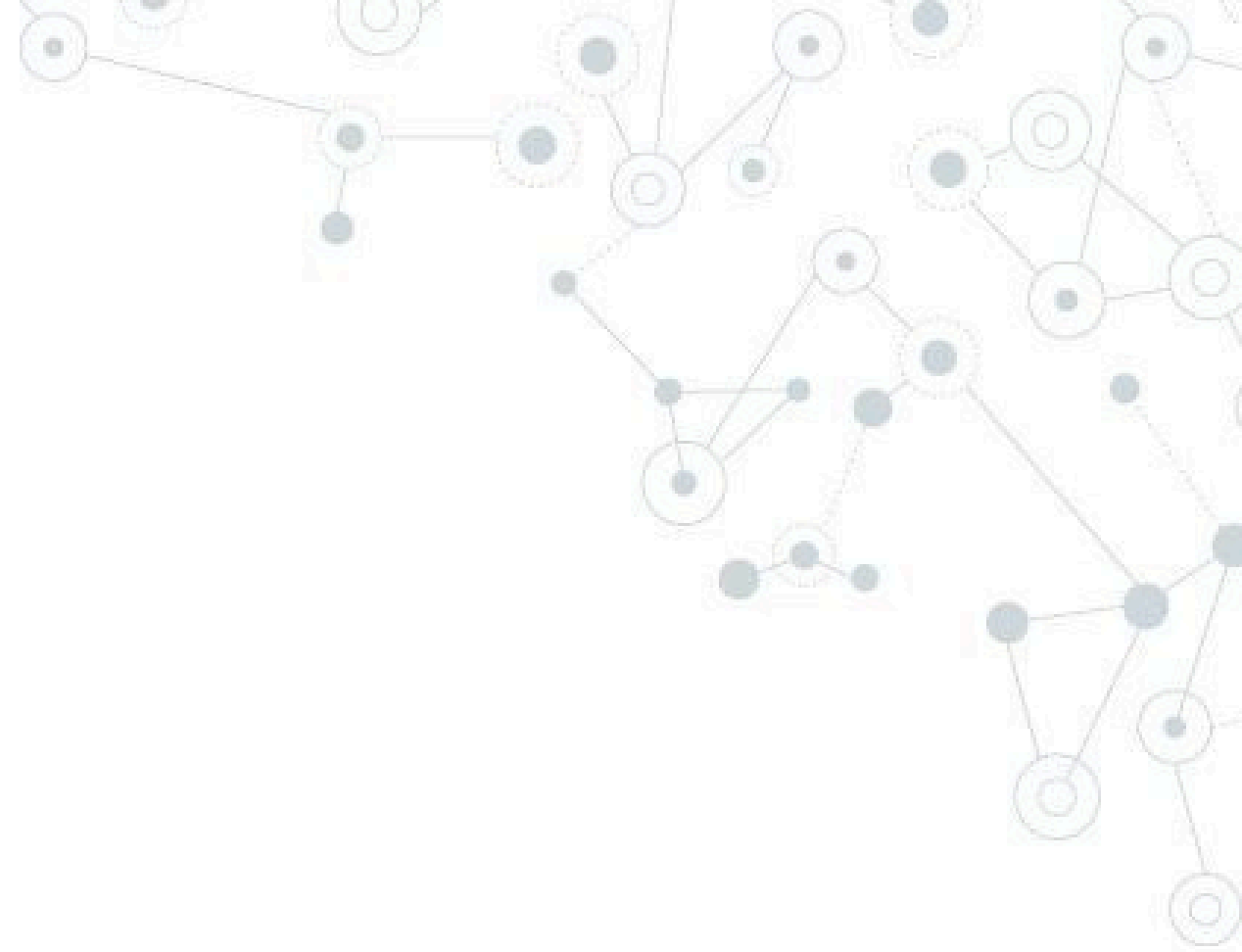
© Files are used to store data

A collection of data or information that are stored on computer known as **file**.

© Different types of files : Text, image, video, audio, binary, csv files

© Files are named location on memory (disk) to store related information

© Since RAM is volatile, there is a need to store data in files for future use.



Without file:

```
name = "Keerthi"
```

```
print(name)
```

Data disappears after program ends

With file:

```
f = open("data.txt", "w")
```

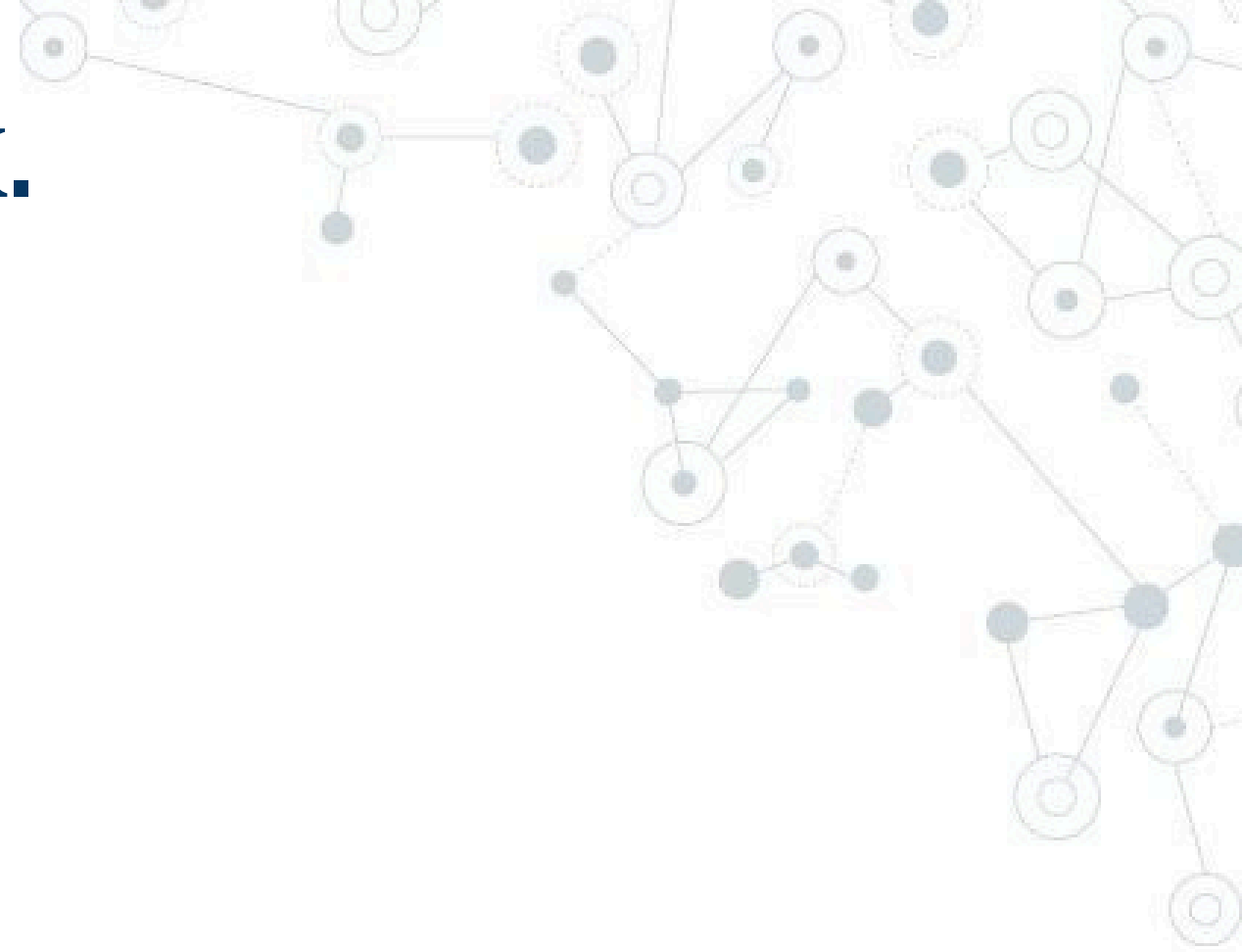
```
f.write("Keerthi")
```

```
f.close()
```

Data is saved in file permanently



A file is a named location on disk used to store data permanently for future use.



Files are storage unit on the hard disk.

Files consist of two things:

i. File name ii) Extension

Both are separated by dot

Ex: program1.py

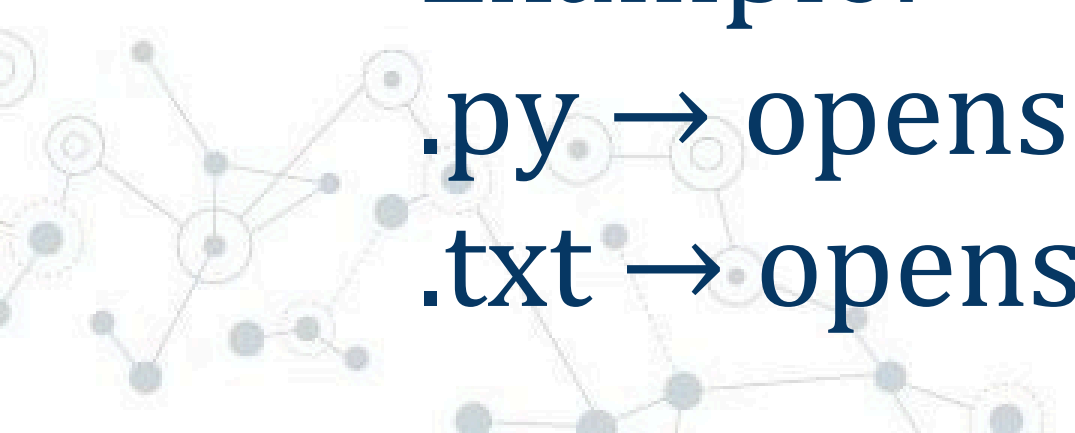
abc.txt

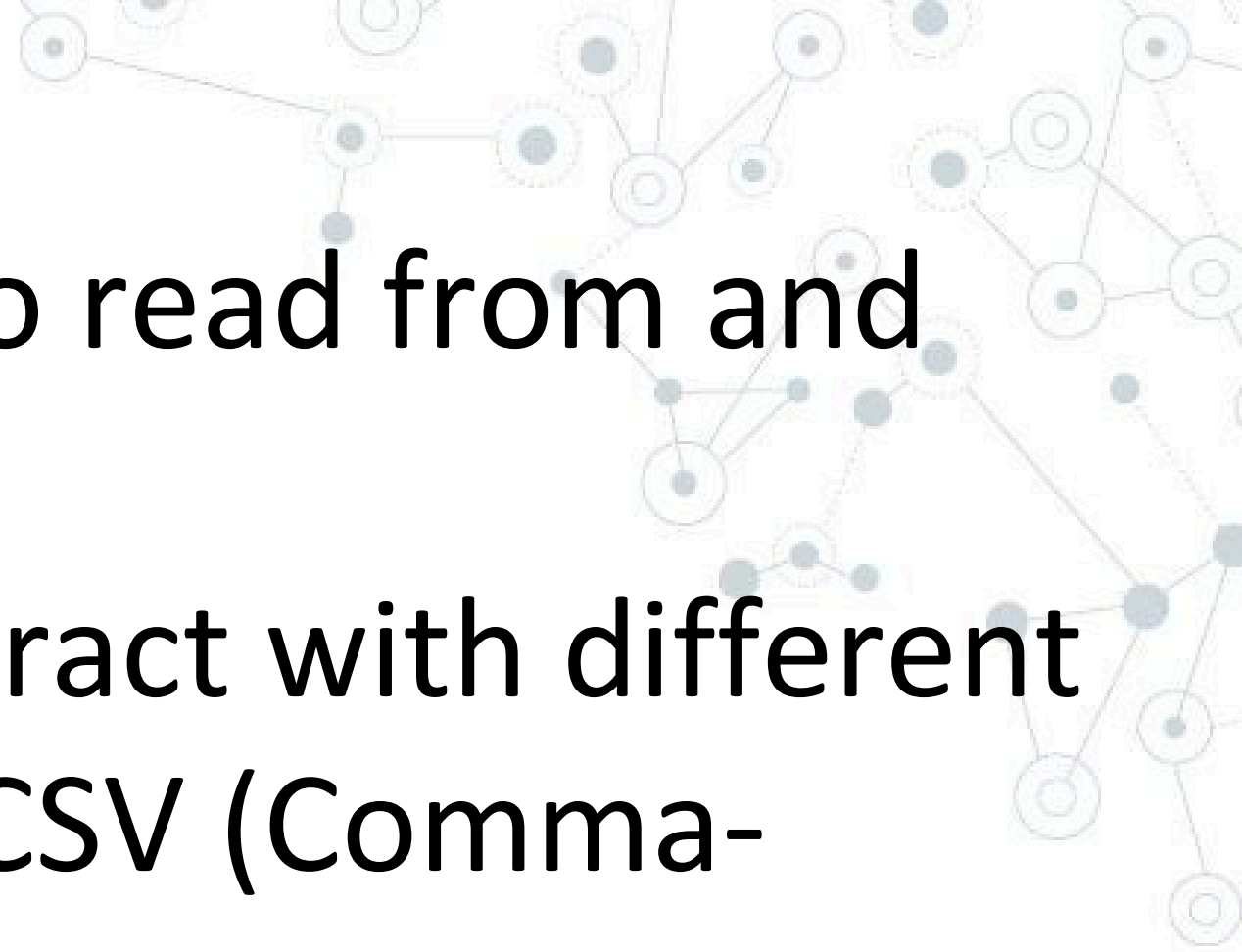
The **extension** helps the computer understand how to open the file

Example:

.py → opens in Python

.txt → opens in Notepad



- 
- File handling in Python allows you to read from and write to files on your computer.
 - Python has built-in functions to interact with different types of files, such as text files and CSV (Comma-Separated Values) files.

File Types:

- **Text files:** These files contain human-readable text.
 - **CSV files:** These are text files where data is stored in a tabular format, separated by commas, which is used for storing structured data like spreadsheets.
- 

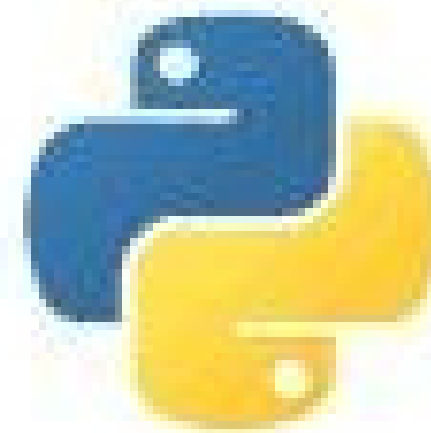
Built-in Functions in Python

Python provides simple functions to work with files:

- `open()` → to open a file
- `read()` → to read data
- `write()` → to write data
- `close()` → to close file

File handling:

- © We can:
- Open a file
 - Write to a file
 - Read from a file
 - Close a file



File handling in Python



File operations:

- © When we want to read from or write to a file we need to open it first.
- © When we are done, it needs to be closed, so that resources that are tied with the file are freed.
- © Hence, in Python, a file operation takes place in the following order.
 - **Open file**
 - **Read or Write**
 - **Close file**

Basic Operations in File Handling

In Python, we mainly perform **4 operations on files:**

Open → Write → Read → Close

Open a File

Before doing anything, we must **open the file**

```
f = open("data.txt", "w")
```

"w" → write mode

Creates file if it does not exist

Write to a File

Used to **store data in the file**

```
f.write("Hello Students")
```

Data is saved into the file

Read from a File

Used to **get data from the file**

```
f = open("data.txt", "r")
```

```
data = f.read()
```

```
print(data)
```

Displays file content

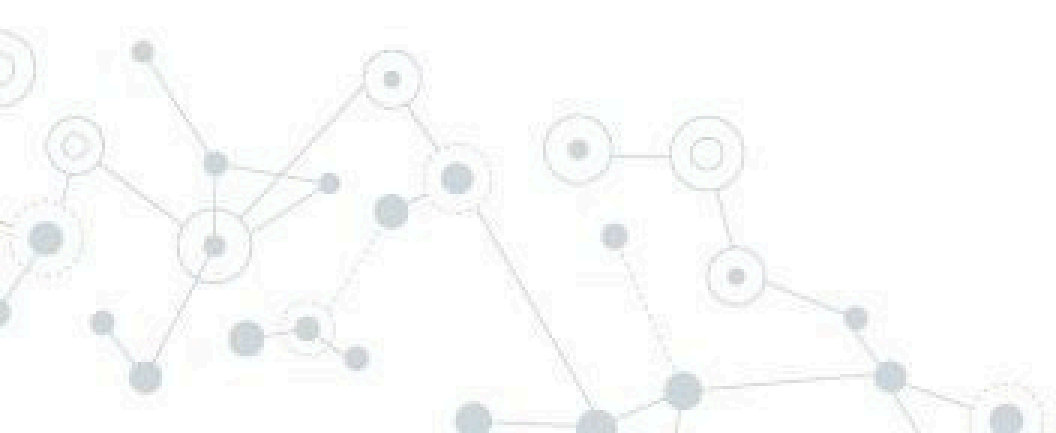
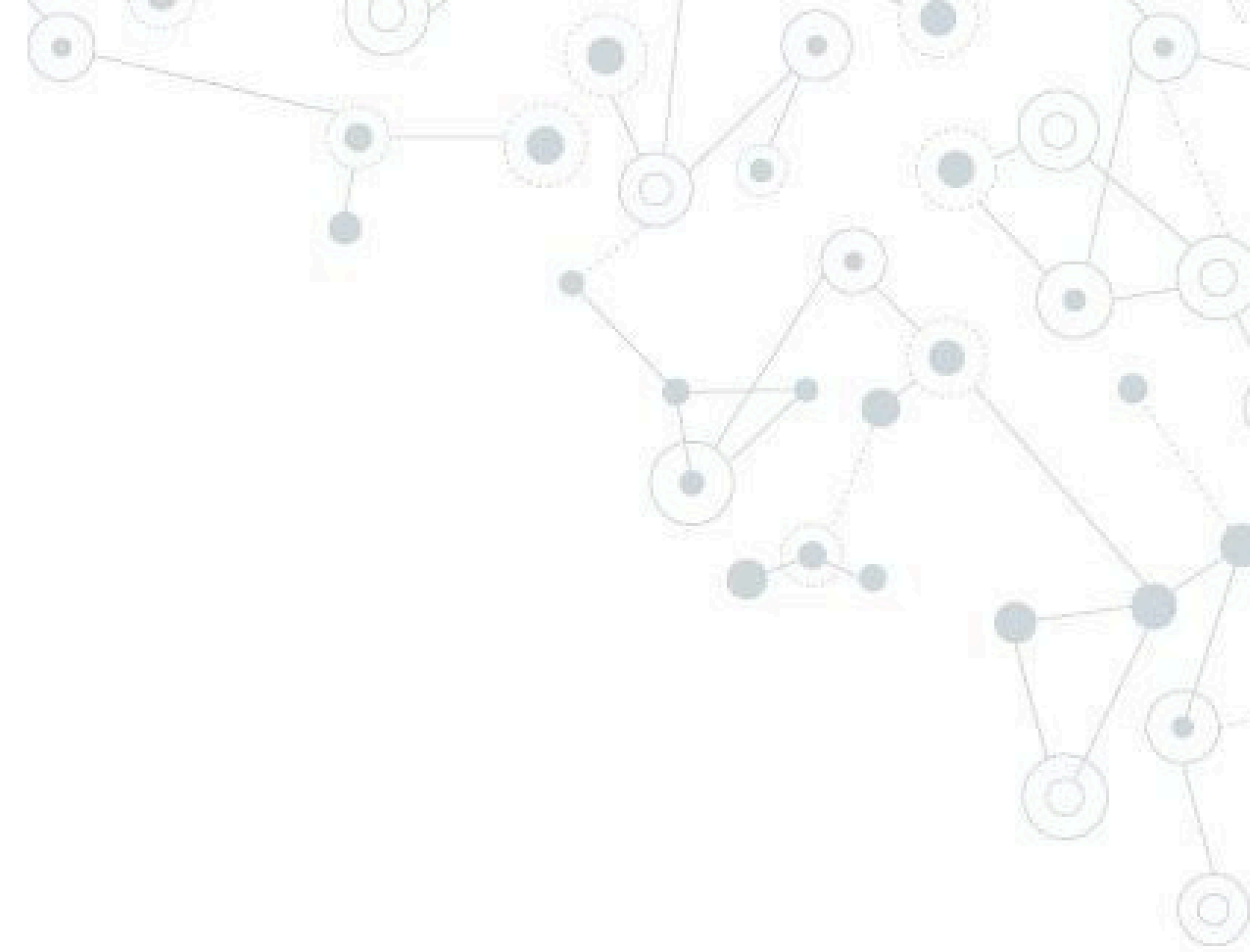
Close a File

After work is done, we must **close the file**

```
f.close()
```

Frees memory

Good programming practice



Text Files

© A text file is a file that contains printable characters and whitespace, organized in to lines separated by newline characters.

© Ex: .txt,
.docx

Working with Text Files

Creating a Text File

To create a new text file, you can use the `open()` function in **write** mode ('w'). If the file doesn't exist, it will be created.

Reading from a Text File

To read the content of a text file, you use the `open()` function in **read** mode ('r').

Writing to a Text File

To write data into a text file, open it in **write** ('w') or **append** ('a') mode. Writing to a file in 'w' mode overwrites the existing content, while 'a' mode adds to the existing content.

Reading and Writing Files

- © We can read a file or write to a file only after creating the file.
- © The file must exist in order to perform any operations on it.
- © Thus python has file operations to open, read, write and close the file

Opening a file

- © Python has a built-in function **open()** to open a file.
- © This function returns a file object, also called a handle, as it is used to read or modify the file accordingly.
- © Syntax:

```
f=open(<filename>,<mode>)
```

```
Ex: f=open("file.txt")
```

f → file object (handle)

This opens a file in current directory (working environment). Default mode is **read**.

What is File Object (Handle)?

It is a reference to the file

Used to perform operations like:

- `f.read()`
- `f.write()`
- `f.close()`

Opening a file

© We can specify full pathname of the file (location where the file is present):

```
f=open("C:/Python3/README.txt")
```

© File can be opened in any modes: read, write
append etc.

Opening a file : file modes

Character	Meaning
'r'	open for reading (default)
'w'	open for writing, truncating the file first
'x'	open for exclusive creation, failing if the file already exists
'a'	open for writing, appending to the end of the file if it exists
'b'	binary mode
't'	text mode (default)
'+'	open a disk file for updating (reading and writing)

Opening a file: Example

1. *`f = open("test.txt")` # equivalent to 'r' or 'rt'*
2. *`f = open("test.txt", 'w')` # write in text mode*
3. *`f = open("img.png", 'r+b')` # read and write in binary mode*

Closing a file

- © When we are done with operations to the file, we need to properly close it.
- © Closing a file will free up the resources that were tied with the file done using the **close()** method.
- © Python has a garbage collector to clean up unreferenced objects but, we must not rely on it to close the file.

Closing a file

What Happens if We Don't Close?

- File may not save properly
- Memory/resources remain occupied
- May cause unexpected errors

Ex:

```
f=open("file.txt")
```

```
# perform file operations like writing to file or  
reading from the file
```

```
f.close()
```

Create a Text File and Write to It

Open a file in write mode, this will create the file if it doesn't exist

```
file = open('example.txt', 'w')
```

Write some text to the file

```
file.write("Hello, this is a text file.\n")
```

```
file.write("This is the second line.")
```

Close the file to save changes

```
file.close()
```

Read from a Text File

```
# Open the file in read mode  
file = open('example.txt', 'r')
```

```
# Read the content of the file  
content = file.read()
```

```
# Print the content of the file  
print(content)
```

```
# Close the file after reading  
file.close()
```

Reading and Writing

◎ REMEMBER:

◎ open function opens an existing file in read mode by default.

◎ Opening a non existing file in write mode will create a new file.

◎ Opening an existing file in write mode will clear the content of the file.

All previous content is erased

New data will replace old data

Append to a Text File (Add More Data)

Open the file in append mode to add more content

```
file = open('example.txt', 'a')
```

Add a new line at the end of the file

```
file.write("\nThis is a new line added to the file.")
```

Close the file after appending

```
file.close()
```

Example: readline()

Sample File

 example - Notepad

File Edit Format View Help

Python Programming Elective.
Department of Civil Engg.
NMAM Institute of Technology, Nitte.
Karkala-574110

Program

```
#readline( )  
myfile=open("example.txt", "r")  
line1 = myfile.readline( )  
line2 = myfile.readline( )  
line3 = myfile.readline(4)  
print(line1, end="")  
print(line2, end="")  
print(line3)
```

Output

Python Programming Elective.
Department of Civil Engg.
NMAM

Example: readlines()

Sample File

 example - Notepad

File Edit Format View Help

```
Python Programming Elective.  
Department of Civil Engg.  
NMAM Institute of Technology, Nitte.  
Karkala-574110
```

Program

```
#readlines( )  
myfile=open("example.txt", "r")  
line1 = myfile.readlines( )  
print(line1)
```

Output

```
['Python Programming Elective.\n', 'Department of Civil Engg.\n',  
'NMAM Institute of Technology, Nitte.\n', 'Karkala-574110']
```

Example: readlines()

Sample File

 example - Notepad

File Edit Format View Help

```
Python Programming Elective.  
Department of Civil Engg.  
NMAM Institute of Technology, Nitte.  
Karkala-574110
```

Program

```
#readlines( )  
myfile=open("example.txt", "r")  
line1 = myfile.readlines( )  
print("The first line is: ", line1[0])  
print("The last line is: ", line1[-1])
```

Output

```
The first line is: Python Programming Elective.
```

```
The last line is: Karkala-574110
```

Program to count number of lines

Sample File

 example - Notepad

File Edit Format View Help

```
Python Programming Elective.  
Department of Civil Engg.  
NMAM Institute of Technology, Nitte.  
Karkala-574110
```

Program

```
#Program to count number of lines  
myfile=open("example.txt", "r")  
a = myfile.readlines( )  
lines = len(a)  
print("The number of lines in the file = ", lines)
```

Output

```
The number of lines in the file = 4
```

Program to find size of the file in bytes

Sample File

 example - Notepad

File Edit Format View Help

```
Python Programming Elective.  
Department of Civil Engg.  
NMAM Institute of Technology, Nitte.  
Karkala-574110
```

Program

```
#Program to find the size of the file in bytes  
myfile=open("example.txt", "r")  
a = myfile.read( )  
size = len(a)
```

Output

```
The size of the file in bytes is = 106
```

Reading a file line by line in python

```
>>> f = open("myfile.txt", 'r')
>>> for line in f:
    print(line)
```

First line.

Second line.

```
>>> f = open("myfile.txt", 'r')
>>> while True:
    line = f.readline()
    if line == "":
        break
    print(line)
```

First line.

Second line.

The **readline** method consumes a line of input and returns this string, including the newline. If **readline** encounters the end of the file, it returns the empty string.

Program to write total marks of students onto a file

```
#program to write marks of students using append "a" mode
myfile=open("marks.txt", "a")
ans='y'
while ans=='y':
    name=input("Enter student 1 name: ")
    sub1=int(input("Enter subject 1 marks: "))
    sub2=int(input("Enter subject 2 marks: "))
    sub3=int(input("Enter subject 3 marks: "))
    total = sub1 + sub2 + sub3
    a = "The total marks of " + name + " is " + str(total) + "\n"
    myfile.write(a)
    ans = input("add next student..?")
myfile.close()
```

Reading Numbers from a File

```
f = open("integers.txt", 'r')
theSum = 0
for line in f:
    line = line.strip()
    number = int(line)
    theSum += number
print("The sum is", theSum)
```

Assume File Content (Example)

integers.txt contains:

10

20

30

Each number is on a **separate line**

Open the file

```
f = open("integers.txt", "r")
```

Opens file in **read mode**

Initialize sum

```
theSum = 0
```

Variable to store total sum

Starts from **0**

Read file line by line

for line in f:

 Reads one line at a time

Execution:

1st → "10\n"

2nd → "20\n"

3rd → "30\n"

Remove newline

line = line.strip()

 Removes \n (newline)

Example:

"10\n" → "10"

Convert to integer

`number = int(line)`

Converts string $\rightarrow$ integer

Example:

"10" $\rightarrow$ 10

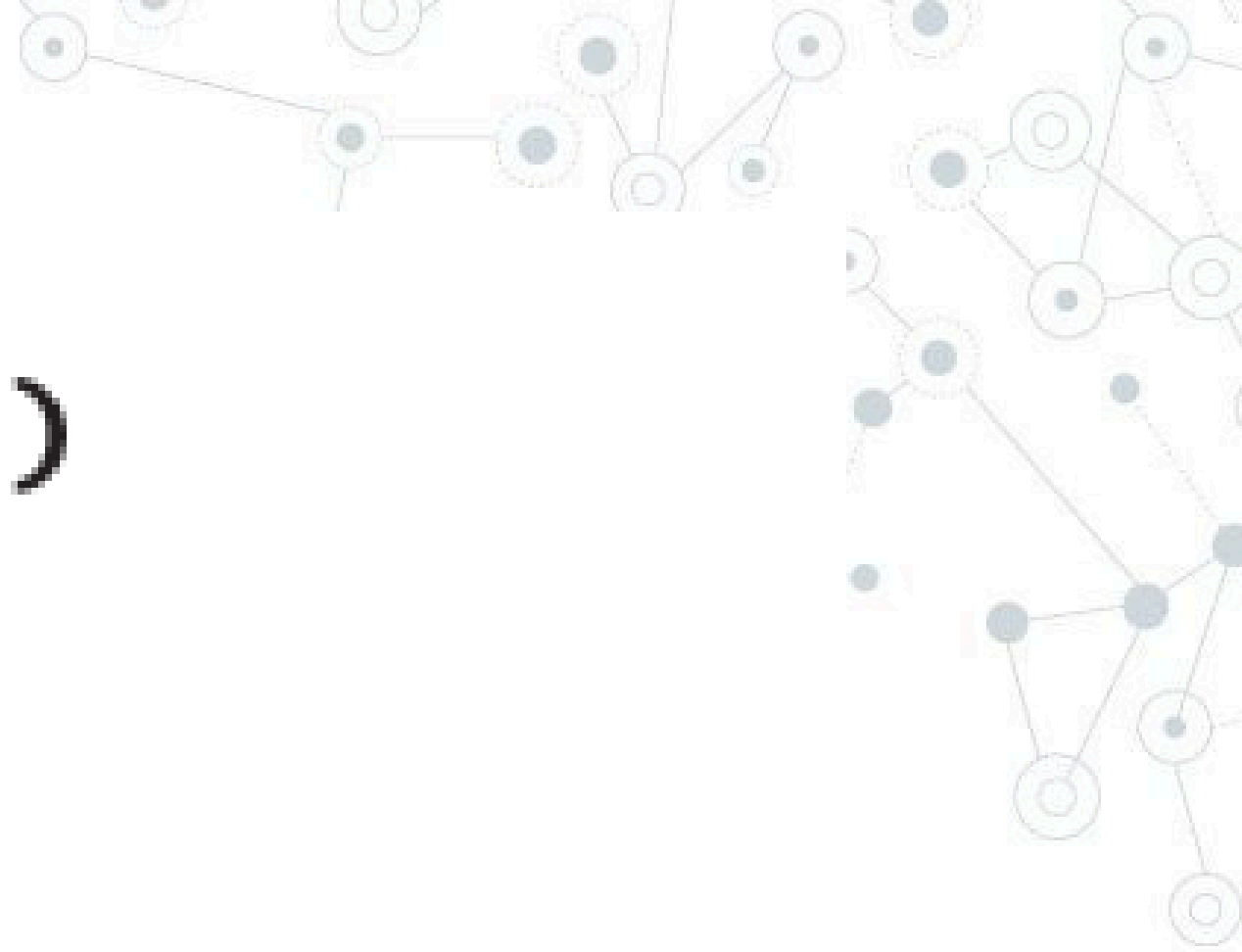
Add to sum

`theSum += number`

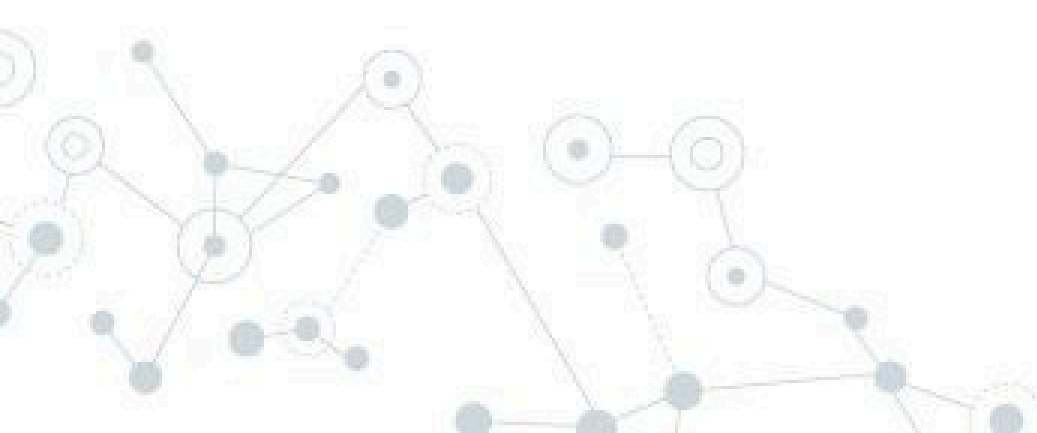
Adds number to total

Step-by-step:

Number	theSum	10	10+20	30+30	60
--------	--------	----	-------	-------	----



```
f = open("integers.txt", 'r')
theSum = 0
for line in f:
    wordlist = line.split()
    for word in wordlist:
        number = int(word)
        theSum += number
print("The sum is", theSum)
```



Assume File Content (Example)

integers.txt contains:

10 20 30

5 15

Open the file

```
f = open("integers.txt", "r")
```

Opens file in **read mode**

File contains numbers

Initialize sum

```
theSum = 0
```

Variable to store total sum

Initially set to **0**

Read file line by line

for line in f:

Reads one line at a time

Execution:

1st line → "10 20 30"

2nd line → "5 15"

Split line into words

wordlist = line.split()

Splits line into list of numbers (strings)

Example:

"10 20 30" → ['10', '20', '30']

"5 15" → ['5', '15']

Loop through each word

for word in wordlist:

Goes through each number one by one

Convert string to integer

number = int(word)

Converts text → number

Example:

'10' → 10

Add to sum

theSum += number

80

Writing number to a file

```
import random
f = open("integers.txt", 'w')
for count in range(500):
    number = random.randint(1, 500)
    f.write(str(number) + '\n')
f.close()
```

CSV File

A CSV (Comma Separated Values) file is a plain text file that stores tabular data, where each line represents a row, and values within a row are separated by commas (or another delimiter). It's used for data exchange, import/export, and storage due to its simplicity and wide compatibility.

Name	Age	Marks
Ravi	20	85
Sneha	21	90

In a **CSV file**, this same data will look like:

Name, Age, Marks

Ravi, 20, 85

Sneha, 21, 90

Working with CSV Files

Creating a CSV File

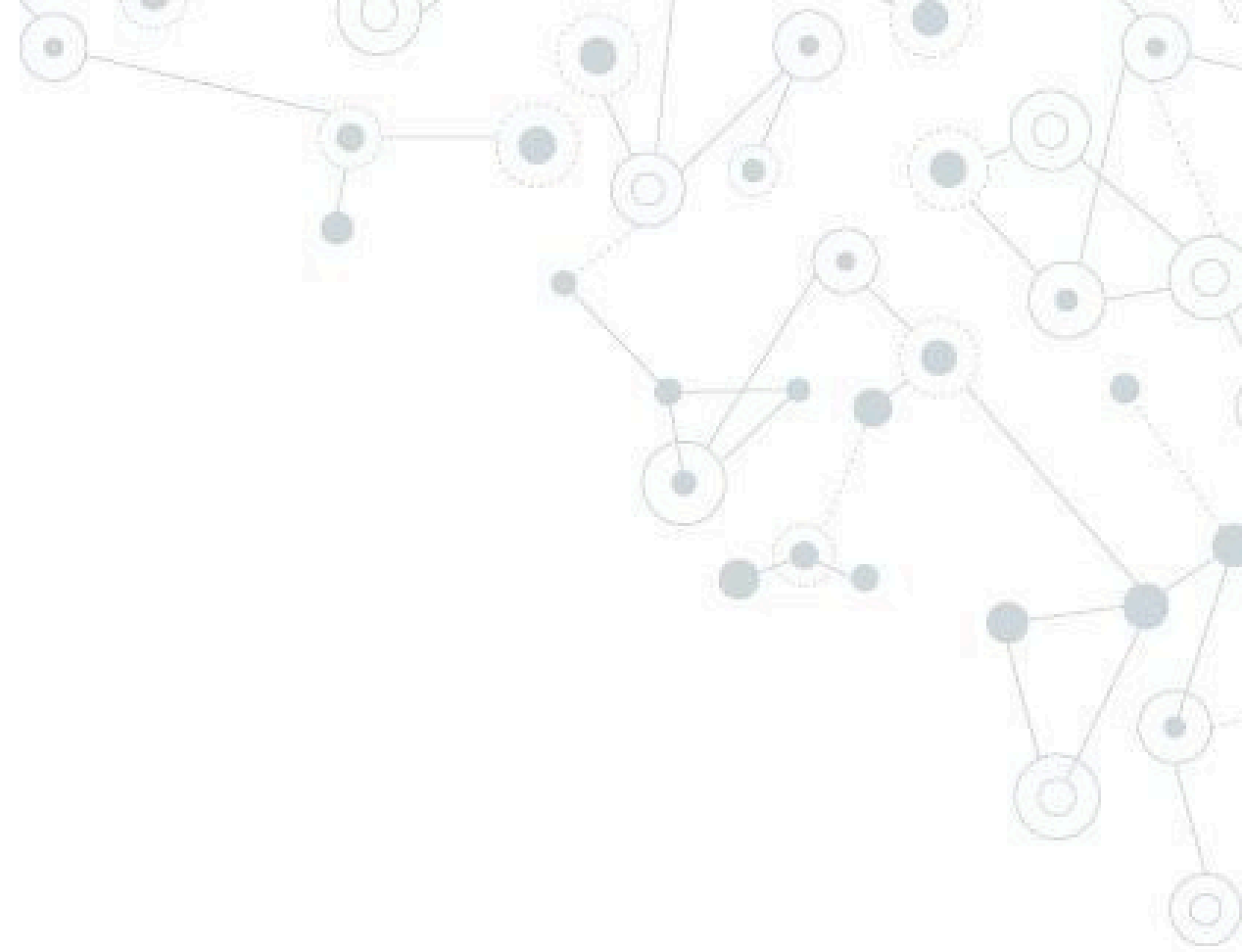
To create a CSV file, you can use Python's built-in `csv` module, which helps you easily write and read tabular data.

Reading a CSV File

The `csv.reader()` method reads data from a CSV file and returns it as rows (each row being a list of values).

Writing to a CSV File

The `csv.writer()` method writes data to a CSV file, with each row being written as a list

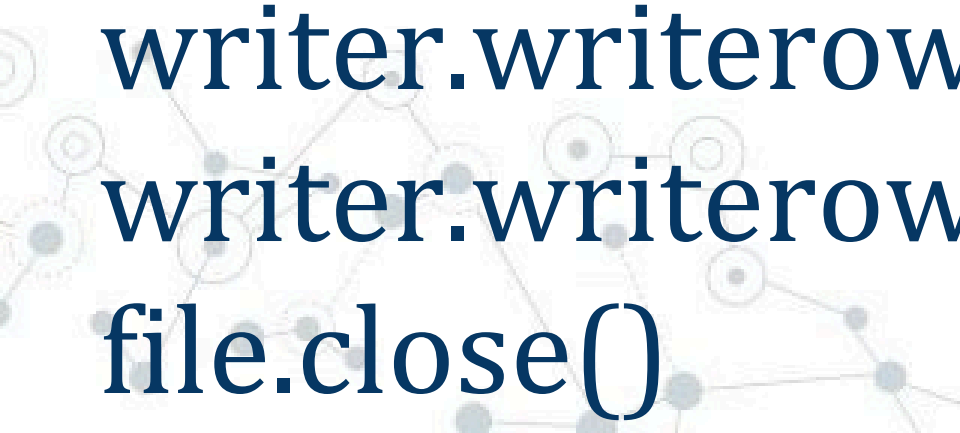


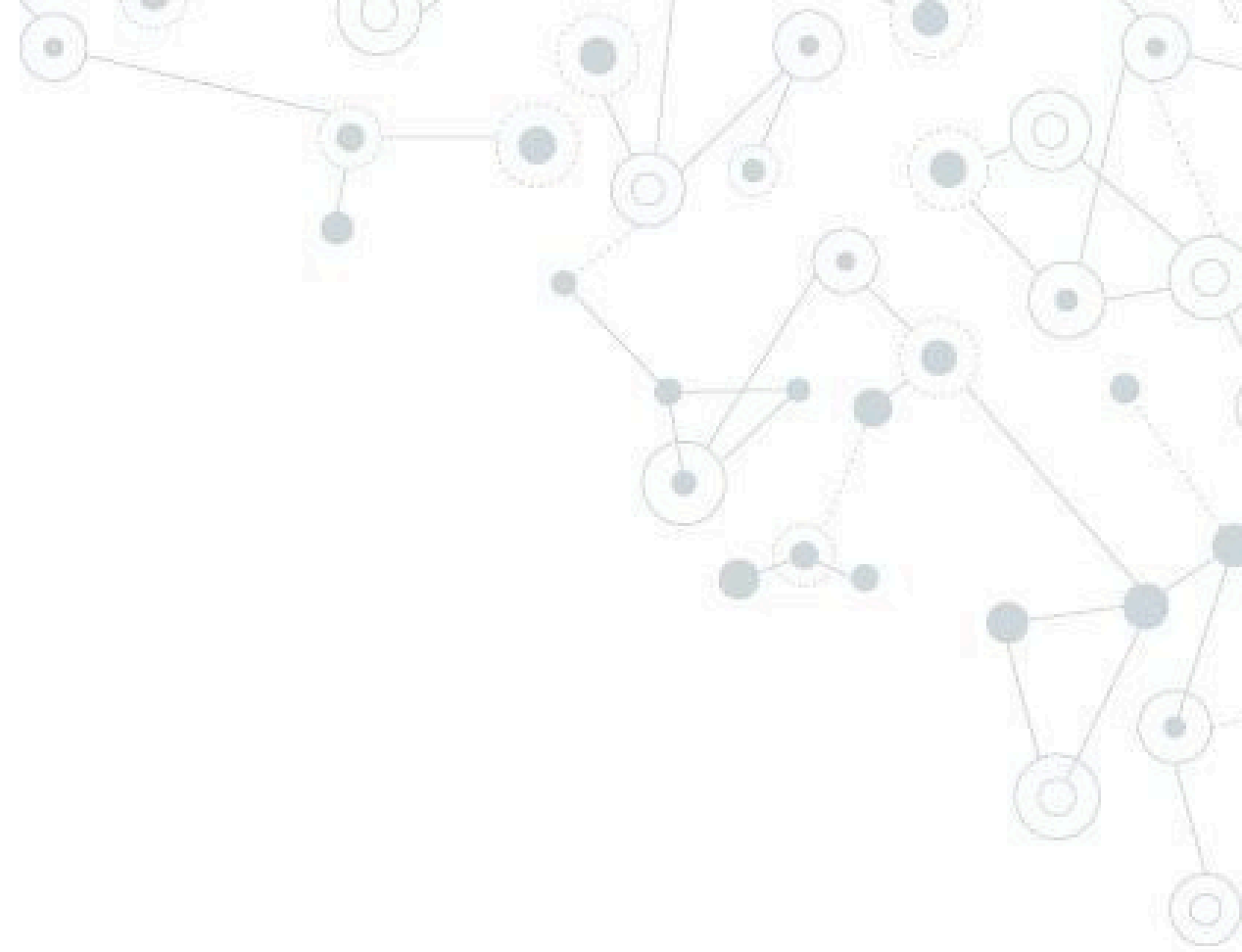
```
import csv
# Open a CSV file in write mode
file = open('data.csv', 'w', newline='')

# Create a CSV writer object
writer = csv.writer(file)

# Writing a header (optional)
writer.writerow(["Name", "Age", "City"])

# Writing data rows
writer.writerow(["Alice", 22, "New York"])
writer.writerow(["Bob", 25, "Los Angeles"])
file.close()
```



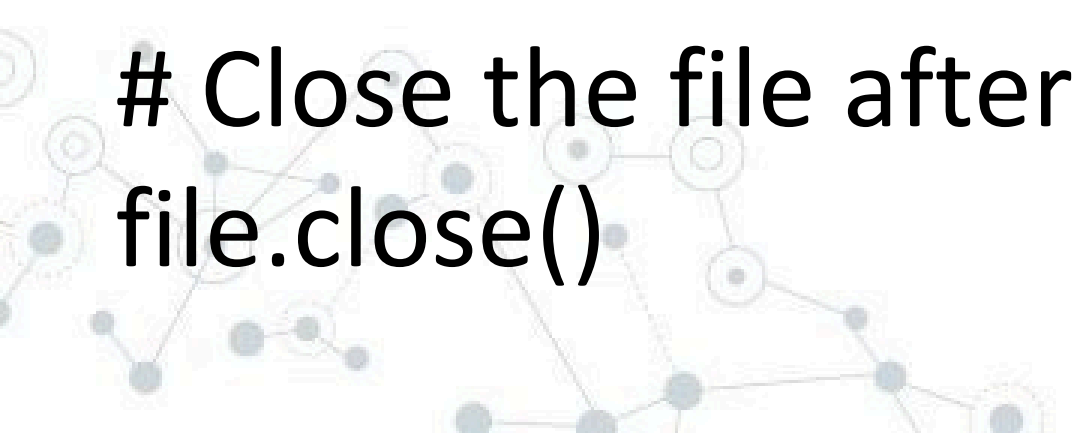


```
import csv
```

```
# Open the CSV file in read mode  
file = open('data.csv', 'r')
```

```
# Create a CSV reader object  
reader = csv.reader(file)
```

```
# Read and print each row  
for row in reader:  
    print(row)
```



```
# Close the file after reading  
file.close()
```

1. Creating & Writing to a CSV File

Python uses the **csv module** to work with CSV files.

Example: Writing data to a CSV file

```
import csv
```

```
# data to write
```

```
data = [  
    ["Name", "Age", "Marks"],  
    ["Ravi", 20, 85],  
    ["Sneha", 21, 90]  
]
```

```
# create and write to CSV file
```

```
with open("students.csv", "w", newline="") as file:
```

```
    writer = csv.writer(file)
```

```
    writer.writerows(data)
```

Explanation

- `import csv` → imports CSV module
- `open("students.csv", "w")` → opens file in **write mode**
- `csv.writer(file)` → creates a writer object
- `writer.writerows(data)` → writes multiple rows

Output file (students.csv)

Name,Age,Marks

Ravi,20,85

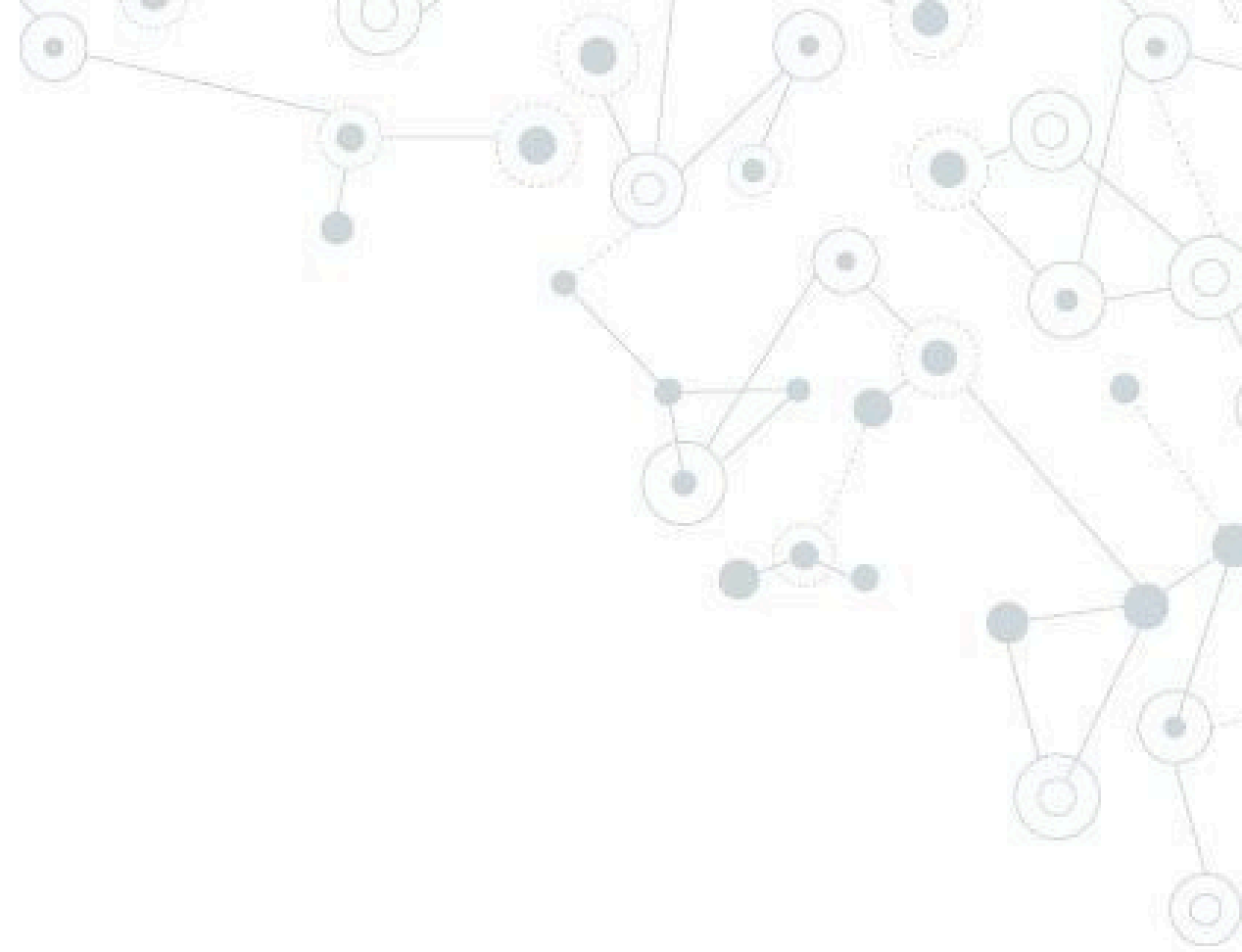
Sneha,21,90

Reading a CSV File:

```
import csv
```

```
with open("students.csv", "r") as file:  
    reader = csv.reader(file)
```

```
for row in reader:  
    print(row)
```



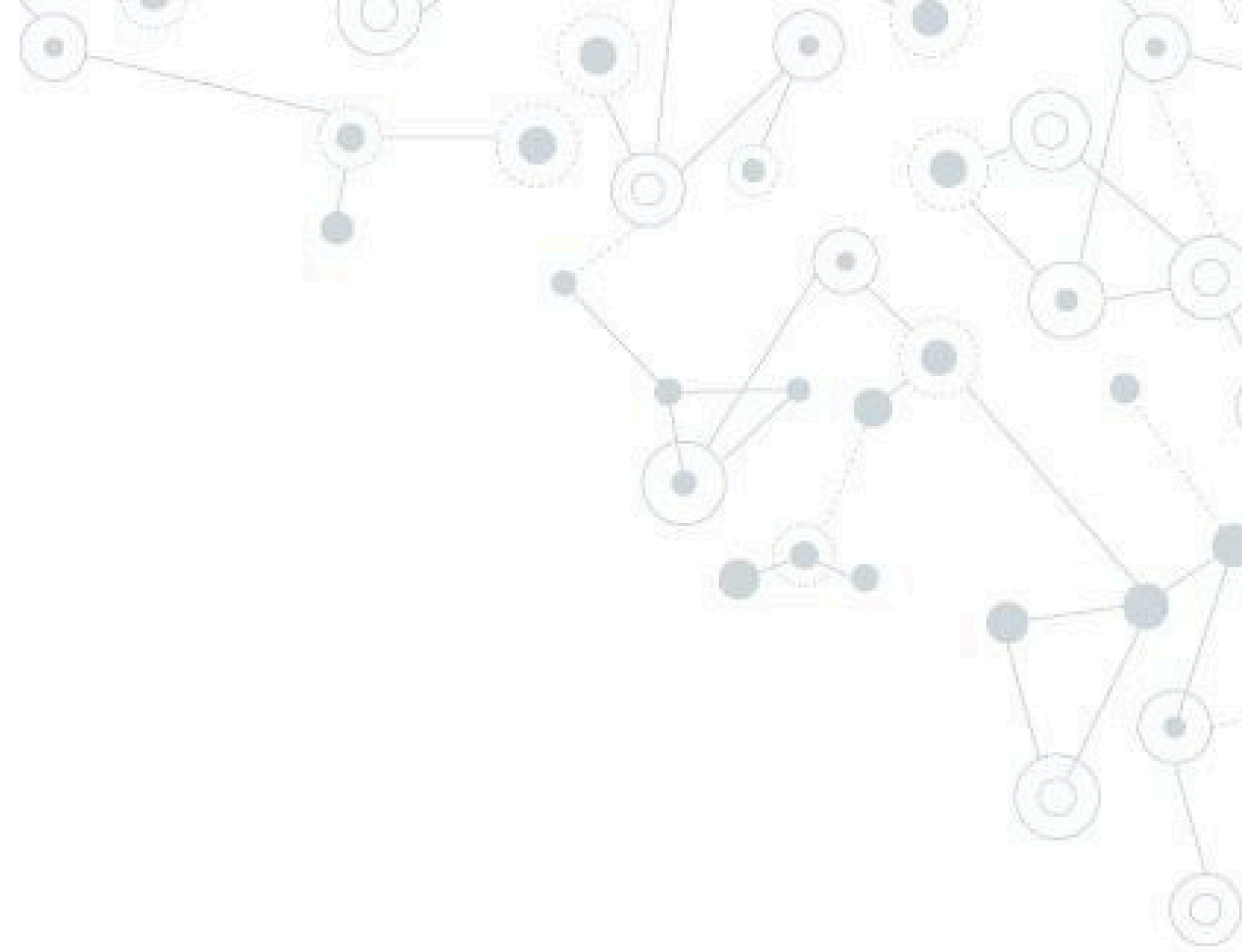
```
import csv
# Open the CSV file in append mode
file = open('data.csv', 'a', newline='')

writer = csv.writer(file)

# Write a new row of data
writer.writerow(["Charlie", 30, "Chicago"])

# Close the file
file.close()
```





```
import csv
```

```
# First sheet: Enter 3 values
```

```
print("Enter 3 values for the first sheet:")
```

```
name = input("Name: ")
```

```
age = input("Age: ")
```

```
city = input("City: ")
```

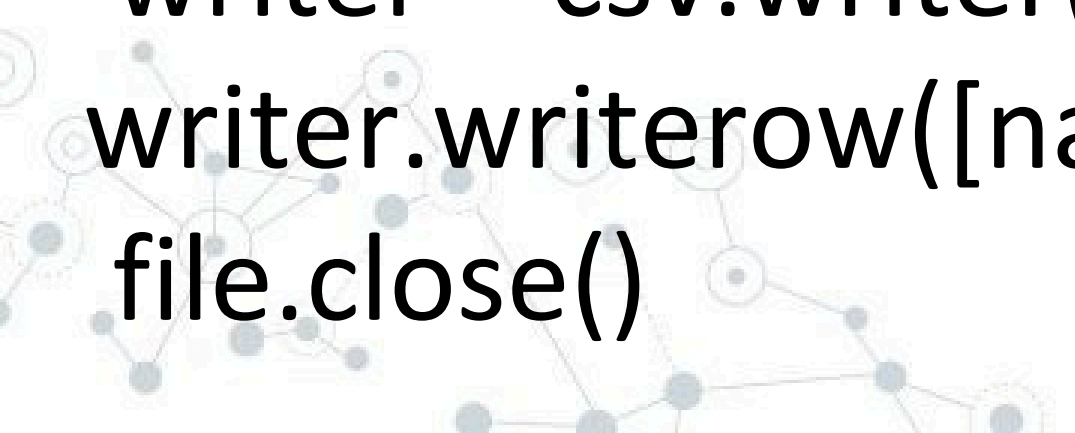
```
# Write to the first CSV file (sheet1.csv)
```

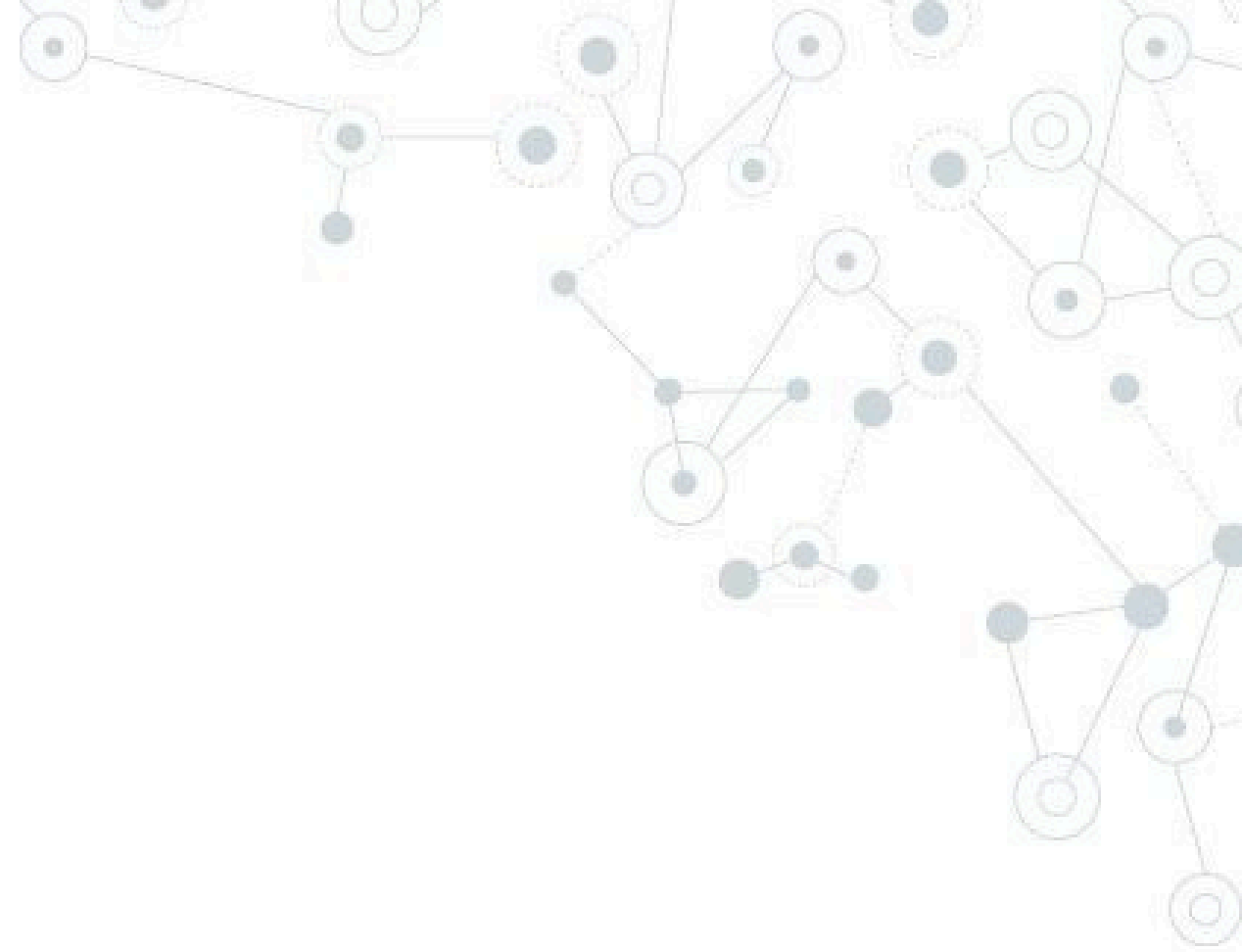
```
file = open('sheet1.csv', 'a', newline='')
```

```
    writer = csv.writer(file)
```

```
    writer.writerow([name, age, city])
```

```
    file.close()
```



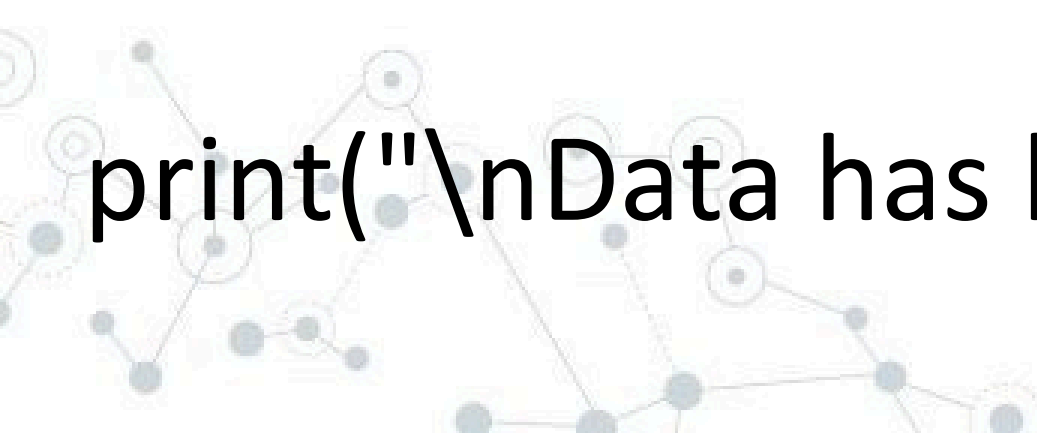


```
# Second sheet: Enter 2 more values
print("\nEnter 2 values for the second sheet:")
country = input("Country: ")
job = input("Job: ")
```

```
# Write to the second CSV file (sheet2.csv)
file = open('sheet2.csv', 'a', newline='')
writer = csv.writer(file)
writer.writerow([country, job])
```

```
file.close()
```

```
print("\nData has been saved to 'sheet1.csv' and 'sheet2.csv'.")
```



Classes and Objects

- Classes and Objects in Python
- Object-Oriented Programming Concepts

What are Classes and Objects?

- World is **object-oriented**
- Full of **objects** like **Sparrow**, **Rose**, **Guitar**, **Keyboard**, etc.
- Each **object** is a **specific instance** of a **class**
- E.g. Sparrow is a specific instance of a **Bird** class
- Rose is a specific instance of a **Flower** class

Examples of Classes and Objects in Real Life

- **Bird** is a class → Sparrow, Crow, Eagle are **objects** of the Bird class.
- **Player** is a class → Sachin, Rahul, Kapil are **objects** of the Player class.
- **Flower** is a class → Rose, Lily, Gerbera are **objects** of the Flower class.
- **Instrument** is a class → Sitar, Flute are **objects** of the Instrument class.

Object-Oriented Thinking

- A class is a **blueprint** or **template** used to create **objects**.
- A **class describes two things**:
 - The **form** an object created from it will take.
 - The **functionality** it will have.
- E.g., a **Bird class** may specify:
- **Form (Properties)**
 - Weight
 - Color
 - Number of feathers

Object-Oriented Thinking

- **Functionality (Methods)**

- Flying
- Hopping
- Chirping
- Eating

Class → Generic in nature

Object → Specific in nature

- A class lets us bundle **data and functionality together**.
- When objects like **Sparrow** or **Eagle** are created from the Bird class, the properties will have specific values.
 - E.g: Property **weight** may be **250 grams** for a Sparrow object.
 - The same property may be **10 kg** for an Eagle object.

Classes and Objects in Programming

- In Python, **every type is a class**.
- Examples of classes: int, str, list, tuple, set, dict
- A class has a **name**, whereas objects are **nameless**
- Since objects do not have names, they are referred to using their **memory addresses**
- Example: num = 10
- A **nameless object** of **type int** is **created** in **memory**, and its **address** is stored in **num**.
- Thus, num refers to the object containing value 10.
- Instead of saying that num refers to a nameless int object, we simply say:
- **num is an int object**

Examples of Classes and Objects in Python

- `a = 3.14` # a is an object of float class
- `s = 'Sudesh'` # s is an object of str class
- `lst = [10, 20, 30]` # lst is an object of list class
- `tpl = ('a', 'b', 'c')` # tpl is an object of tuple class
- Different objects of the same type may contain **different data but the same methods.**
- Example:
- `s1 = 'Rupesh'` # s1 is an object of type str
- `s2 = 'Geeta'` # s2 is an object of type str

Examples of Objects

- `s1 = 'Rupesh'` # s1 is an object of type str
- `s2 = 'Geeta'` # s2 is an object of type str
- s1 and s2 contain **different data**
- But both support the same methods such as:
 - `upper()`
 - `lower()`
 - `capitalize()`
- The specific data stored in an object is called:
- **Instance data**
- **Attributes**
- **State of the object**
- The functions defined inside objects are called **methods**.

User-defined Classes

- In addition to providing ready-made classes like **int**, **str**, **list**, **tuple**, Python also allows us to **define our own classes** and create objects from them.
- These are called **user-defined classes**.
- The rules for defining and using user-defined classes are the same as for standard classes.

Class vs Object

- **Class** → Blueprint
- **Object** → Instance created from the class
- The process of creating an object from a class is called **instantiation**
- **Attributes** store **data**
- **Methods** perform **operations** on that data

OOP, Defining a Class

- Python was built as a procedural language
 - OOP exists and works fine, but feels a bit more "tacked on"
- Declaring a class:

```
class name:  
    statements
```

Fields

name = value

– Example:

```
class Point:  
    x = 0  
    y = 0
```

```
# main  
p1 = Point()  
p1.x = 2  
p1.y = -5
```

point.py

```
1 class Point:  
2     x = 0  
3     y = 0
```

Using a Class

`import class`

– client programs must import the classes they use

point main.py

```
1  from Point import *
2
3  # main
4  p1 = Point()
5  p1.x = 7
6  p1.y = -3
7  ...
8
9  # Python objects are dynamic (can add fields any time!)
10 p1.name = "Tyler Durden"
```

- In Python, you can add new attributes anytime!
- Even if name was NOT defined in the class, you can still add it:

Now p1 has:

➤ x

➤ y

➤ name

Object Methods

```
def name (self, parameter, ..., parameter) :  
    statements
```

- self refers to the current object
- It is similar to this in C++/Java
- **self** *must* be the first parameter to any object method
- *must* access the object's fields through the `self` reference
- Each object stores its own data separately

```
class Point:  
    def translate(self, dx, dy):  
        self.x += dx  
        self.y += dy  
    ...
```

Write distance, set_location, and distance_from_origin methods.

point.py

```
1  from math import *
2
3  class Point:
4      x = 0
5      y = 0
6
7      def set_location(self, x, y):
8          self.x = x
9          self.y = y
10
11     def distance_from_origin(self):
12         return sqrt(self.x * self.x + self.y * self.y)
13
14     def distance(self, other):
15         dx = self.x - other.x
16         dy = self.y - other.y
17         return sqrt(dx * dx + dy * dy)
```

```
class Employee:
    def set_data(self, n, a, s):
        self.name = n
        self.age = a
        self.salary = s

    def display_data(self):
        print(self.name, self.age, self.salary)
```

```
e1 = Employee()
e1.set_data('Ramesh', 23, 25000)
e1.display_data()
```

```
e2 = Employee()
e2.set_data('Suresh', 25, 30000)
e2.display_data()
```